Practical Task 4.2

(Credit Task)

Submission deadline: Monday, April 27
Discussion deadline: Friday, May 15

General Instructions

This practical task focuses on the ***program design***, mainly the ***functional composition*** and the ***code reusability***, and on the ***composition*** as a type of ***class relationships***.

In the first part, you will need to examine an existing source code to identify ***duplicate code***, which means that there are sequences of commands that occur more than once. Duplicate code is generally considered undesirable for a number of reasons, for example:

- Duplicate code makes your program lengthy and bulky.
- Duplication makes ***software maintenance*** more difficult.
- Duplication decreases your ***code quality***. Code quality is a necessity to make your software survive for long. Having duplicate code will make your code smelly and increases the ***technical debt*** associated with it. The cost of repair of this debt is the amount of capital and time required to pay to a developer to simplify or de-duplicate it.
- Duplication leads to so-called ***Shotgun Surgery***: Suppose you wrote a buggy code and have to do a ***code review*** to find out the issue and then fix it. You now have to fix every location of that code, losing your time, efficiency and sometimes temper.
- Duplicate code increases security risks: When someone takes a code from somewhere and adds into other parts of program, they might forget about the holes and endings that the plagiarized code has.

This issue related to the given code will require you to work on the function composition, which implies that complex problems should be solved by decomposing them into many smaller problems so that each can be worked out easily. Eventually, those small pieces have to be put together to form the overall solution. One way of combining these small pieces is the function composition, a mechanism to combine simple functions to build more complicated ones. Furthermore, the function composition is a great tool that makes the code more compact and reduces noise. Because of the concise syntax, there are fewer possibilities to make mistakes like, for example, mixing up parameters.

In the second part, your task is to extend the program by adding new features and functionality. To make the design efficient, you should use a mechanism known as the ***class composition***, the most commonly used class relationship. In general terms, composition allows a class to contain an object instance of another class. Composition can be denoted as being an "***as a part***" or a "***has a***" ***relationship*** between classes. Thus, the main class becomes the front-end class, and the other represents the back-end class. The composition approach provides strong ***encapsulation*** because a change to a back-end class does not necessarily break any code that relies on the front-end class. Thus, it maintains the same access interface to the front-end despite of back-end changes.

Now, let's move to the task. Imagine, as it commonly happens in the IT world, that you have been hired by BuggySoft, a leading producer of buggy software, to replace one of their staff who is leaving. You are asked to continue working on a simple Task Planner. They assure you that the task is very simple and should not take more than a couple of hours because the program looks fine from the perspective of the user interface. So, they just want you to extend the program a bit as their customer wants to add some additional features. You agree, and …

1. Open the source code file attached to this practical task. Suppose that this is the code inherited from the previous programmer. As you can see, the code itself is not large, but everything is placed in a single `Main` method. This code is messy and not documented, but you are lucky because you at least can compile and run it.

   You now must spend some time to understand how the code works, what it produces, and how it interacts with the user. The best way to do this is to create a new C# Console Application project and import the given file. You then should run the code step-by-step in the debug mode to explore the control flow.

2. To demonstrate that you are a more experienced programmer than your leaving colleague, you should ensure the function composition and avoid any code duplication because of the aforementioned reasons. Find all sequences of commands that should be placed together to form logically complete methods parametrised with a set of arguments. Work on this to make your code as clean as possible. In addition, add comments so that your methods and key points of your program make sense in case you want to return to it later.

   Once finished, save your source code file as '`RevisedCode.cs`'. You will need to submit this as part of your solution.

3. As you can see, this is a Task Planner program that partitions tasks into a number of categories. The tasks in a same category form a priority list. All tasks are printed in a table format. The program provides a very limited functionality and is for sure not competitive. Because of this, the customer requested BuggySoft to add the following capabilities:
   - The user must be given an option to add new and delete existing categories. If a category is removed, all associated tasks should be removed as well.
   - The user must be given an option to delete an existing task.
   - It must be possible to move a task in its respective priority list by setting the place (priority).
   - It must be possible to move a task from one category to another.
   - It must be possible to highlight important tasks so that they appear in red in the terminal.
   - Every task must be characterised by a due date printed along with the name. This might need to enlarge the size of the table's columns.

   From what you see, the customer does not specify a particular design and leaves this with you. You, as a programmer, are also free to implement the internal logic of the program as you decide. However, a proper design will likely rely on ***composition***. For example, you might have already noticed that tasks are to be "***a part of***" category and they are deleted along with their category. Remember that the class composition ensures the ***whole/part*** or ***parent/child relationship***; that is, the life cycle of the part (or child) is controlled by the whole (or parent) that owns it. If the parent object is destroyed, then the child objects also cease to exist.

   Note that you will likely need to create new classes to manipulate objects as data structures. For example, each task is characterised by a colour tag (to stress its importance) and a due date. This is not currently presented in the existing source code as well as any classes but the main class. Because we do not know how many classes you wish to create, we leave it with you and expect you to add them together to the '`FinalCode.cs`', which is to be the only file that you must submit as your final solution.

   In addition, you will deal with a collection of objects to represent categories and task priority lists. Therefore, we recommend you to focus on the use of dynamic `List<T>` data structure rather than a quite static array. You are also allowed to alter the existing code in order to improve it.

   Finally, you will have to significantly extend the existing menu to interact with the user to serve incoming requests.

Remember that we do understand that there is no unique correct solution to this problem. We expect you to provide us with a well-documented, neatly written code that has a justified program design.

Further Notes

- Study the connection between the object relationships and the class relationships, their differences, and how to implement them in your programs by reading Section 4.2 of SIT232 Workbook available in CloudDeakin → Learning Resources.
- The following links will give you more insights about the program design aspects related to this practical task:
  - What is a duplicate code? (https://www.codegrip.tech/productivity/what-is-duplicate-code/)
  - Code smell (https://en.wikipedia.org/wiki/Code_smell)
  - Everything you need to know about Code Smells (https://www.codegrip.tech/productivity/everything-you-need-to-know-about-code-smells/)
- The following links will give you more ideas about the class/object relationships, and specifically about composition:
  - https://creately.com/blog/diagrams/class-diagram-relationships/
  - https://www.visual-paradigm.com/guide/uml-unified-modeling-language/uml-aggregation-vs-composition/
  - https://www.c-sharpcorner.com/UploadFile/ff2f08/association-aggregation-and-composition/

Submission Instructions and Marking Process

To get your task completed, you must finish the following steps strictly on time.

- Make sure your programs implement the required functionality. They must compile and have no runtime errors. Programs causing compilation or runtime errors will not be accepted as a solution. You need to test your programs thoroughly before submission. Think about potential errors where your programs might fail.
- **Submit** the expected code files as a solution to the task via OnTrack submission system.
- Once your solution is accepted by your tutor, you will be invited to **continue its discussion and answer relevant theoretical questions through a face-to-face interview**. Specifically, you will need to meet with the tutor to demonstrate and discuss the solution in one of the dedicated practical sessions (run online via MS Teams for cloud students and on-campus for students who selected to join classes at Burwood). Please, come prepared so that the class time is used efficiently and fairly for all students in it. Be on time with respect to the specified discussion deadline.

  You will also need to **answer all additional questions** that your tutor may ask you. Questions will cover the lecture notes; so, attending (or watching) the lectures should help you with this **compulsory** discussion part. You should start the discussion as soon as possible as if your answers are wrong, you may have to pass another round, still before the deadline. Use available attempts properly.

Note that we will not accept your solution after **the submission deadline** and will not discuss it after **the discussion deadline**. If you fail one of the deadlines, you fail the task and reduce the chance to obtain the target grade. Unless extended for all students, the deadlines are strict to guarantee smooth and on-time work throughout the unit.

Remember that this is your responsibility to keep track of your progress in the unit that includes checking which tasks have been marked as completed in the OnTrack system by your marking tutor, and which are still to be finalised. When marking you at the end of the unit, we will solely rely on the records of the OnTrack system and feedback provided by your tutor about your overall progress and the quality of your solutions.